

In-Class Assignment: Integrate Ticket Service with AI Selector and SQS Producer

Goal

Update the Ticket Service so that `POST /tickets` can:

1. Create a ticket.
2. Call the LLM workflow selector.
3. If the selector result is ready, create a workflow run.
4. Attach the workflow run to the ticket.
5. Send the `workflow_run_id` to SQS.
6. Update ticket and workflow run statuses correctly.

Final successful flow:

```
POST /tickets
|
v
Create Ticket
Ticket.status = CREATED
|
v
Call LLM Workflow Selector
|
v
Create WorkflowRun
WorkflowRun.status = CREATED
|
v
Attach WorkflowRun to Ticket
Ticket.status = WORKFLOW_CREATED
|
v
Send SQS Message
{
  "workflow_run_id": "run_xxxxxxx"
}
|
v
Update Status
WorkflowRun.status = QUEUED
Ticket.status = WORKFLOW_QUEUED
```

Step 1: Create WorkflowRuns DynamoDB Table

Open AWS Console.

Go to:

```
DynamoDB
→ Tables
→ Create table
```

Use this configuration:

```
Table name: WorkflowRuns
Partition key: workflow_run_id
Partition key type: String
Sort key: None
Table settings: Default settings
Capacity mode: On-demand
```

Wait until table status becomes:

```
Active
```

Step 2: Create SQS Queue

Open AWS Console.

Go to:

```
SQS
→ Create queue
```

Use this configuration:

```
Type: Standard
Name: ticket-workflow-queue
```

```
Visibility timeout: 30 seconds
Message retention period: Default
```

After creating the queue, copy the Queue URL.

Step 3: Set Environment Variables

Run:

```
export AWS_REGION="us-west-2"
export TICKETS_TABLE_NAME="Tickets"
export WORKFLOW_DEFINITIONS_TABLE_NAME="WorkflowDefinitions"
export WORKFLOW_RUNS_TABLE_NAME="WorkflowRuns"
export WORKFLOW_QUEUE_URL="YOUR_SQS_QUEUE_URL"
export OPENAI_API_KEY="YOUR_OPENAI_API_KEY"
export OPENAI_MODEL="gpt-5"
```

Step 4: Update requirements.txt

```
fastapi
uvicorn
pydantic
boto3
openai
```

Run:

```
pip install -r requirements.txt
```

Step 5: Update app/config.py

```
import os

AWS_REGION = os.getenv("AWS_REGION", "us-west-2")
```

```

TICKETS_TABLE_NAME = os.getenv("TICKETS_TABLE_NAME", "Tickets")

WORKFLOW_DEFINITIONS_TABLE_NAME = os.getenv(
    "WORKFLOW_DEFINITIONS_TABLE_NAME",
    "WorkflowDefinitions",
)

WORKFLOW_RUNS_TABLE_NAME = os.getenv(
    "WORKFLOW_RUNS_TABLE_NAME",
    "WorkflowRuns",
)

WORKFLOW_QUEUE_URL = os.getenv("WORKFLOW_QUEUE_URL", "")

OPENAI_MODEL = os.getenv("OPENAI_MODEL", "gpt-5")

```

Step 6: Update Ticket Models

Open:

```
app/models/ticket_models.py
```

Replace it with:

```

from enum import Enum
from typing import Dict, List, Optional

from pydantic import BaseModel, Field

class TicketStatus(str, Enum):
    CREATED = "CREATED"
    WORKFLOW_SELECTED = "WORKFLOW_SELECTED"
    WAITING_FOR_INPUT = "WAITING_FOR_INPUT"
    WORKFLOW_CREATED = "WORKFLOW_CREATED"
    WORKFLOW_QUEUED = "WORKFLOW_QUEUED"
    WORKFLOW_RUNNING = "WORKFLOW_RUNNING"
    RESOLVED = "RESOLVED"
    FAILED = "FAILED"

class CreateTicketRequest(BaseModel):
    user_id: str = Field(..., min_length=1)

```

```
message: str = Field(..., min_length=5)
source: Optional[str] = "unknown"
```

```
class UpdateTicketStatusRequest(BaseModel):
    status: TicketStatus
```

```
class TicketResponse(BaseModel):
    ticket_id: str
    user_id: str
    message: str
    source: str
    status: TicketStatus
    created_at: str
    updated_at: str

    selected_workflow_id: Optional[str] = None
    selected_workflow_type: Optional[str] = None
    workflow_run_id: Optional[str] = None

    extracted_fields: Dict[str, str] = {}
    missing_fields: List[str] = []
    missing_field_questions: List[str] = []
```

Step 7: Create Workflow Run Models

Create:

```
app/models/workflow_run_models.py
```

Add:

```
from enum import Enum
from typing import Dict, Optional

from pydantic import BaseModel

class WorkflowRunStatus(str, Enum):
    CREATED = "CREATED"
    QUEUED = "QUEUED"
    RUNNING = "RUNNING"
```

```
COMPLETED = "COMPLETED"
FAILED = "FAILED"
```

```
class WorkflowRunResponse(BaseModel):
    workflow_run_id: str
    ticket_id: str
    workflow_id: str
    workflow_type: str
    status: WorkflowRunStatus
    workflow_input: Dict[str, str]
    created_at: str
    updated_at: str
    error_message: Optional[str] = None
```

Step 8: Update Ticket Repository

Open:

```
app/repositories/ticket_repository.py
```

Replace it with:

```
from typing import List, Optional

import boto3
from botocore.exceptions import ClientError

from app.config import AWS_REGION, TICKETS_TABLE_NAME
from app.models.ticket_models import TicketResponse, TicketStatus

class TicketRepository:
    def __init__(self):
        dynamodb = boto3.resource("dynamodb", region_name=AWS_REGION)
        self.table = dynamodb.Table(TICKETS_TABLE_NAME)

    def save(self, ticket: TicketResponse) -> TicketResponse:
        self.table.put_item(Item=self._ticket_to_item(ticket))
        return ticket

    def get_by_id(self, ticket_id: str) -> Optional[TicketResponse]:
        response = self.table.get_item(
```

```

        Key={
            "ticket_id": ticket_id
        }
    )

    item = response.get("Item")

    if item is None:
        return None

    return self._item_to_ticket(item)

def list_all(self) -> List[TicketResponse]:
    response = self.table.scan()
    items = response.get("Items", [])

    return [
        self._item_to_ticket(item)
        for item in items
    ]

def update_status(
    self,
    ticket_id: str,
    status: TicketStatus,
    updated_at: str,
) -> Optional[TicketResponse]:
    try:
        response = self.table.update_item(
            Key={
                "ticket_id": ticket_id
            },
            UpdateExpression="SET #status = :status, updated_at
= :updated_at",
            ConditionExpression="attribute_exists(ticket_id)",
            ExpressionAttributeNames={
                "#status": "status"
            },
            ExpressionAttributeValues={
                ":status": status.value,
                ":updated_at": updated_at,
            },
            ReturnValues="ALL_NEW",
        )

        return self._item_to_ticket(response["Attributes"])

    except ClientError as error:

```

```

        if error.response["Error"]["Code"] ==
"ConditionalCheckFailedException":
            return None
        raise

def mark_waiting_for_input(
    self,
    ticket_id: str,
    selected_workflow_id: Optional[str],
    selected_workflow_type: Optional[str],
    extracted_fields: dict,
    missing_fields: list,
    missing_field_questions: list,
    updated_at: str,
) -> Optional[TicketResponse]:
    try:
        response = self.table.update_item(
            Key={
                "ticket_id": ticket_id
            },
            UpdateExpression=(
                "SET #status = :status, "
                "selected_workflow_id = :selected_workflow_id, "
                "selected_workflow_type = :selected_workflow_type, "
                "extracted_fields = :extracted_fields, "
                "missing_fields = :missing_fields, "
                "missing_field_questions = :missing_field_questions, "
                "updated_at = :updated_at"
            ),
            ConditionExpression="attribute_exists(ticket_id)",
            ExpressionAttributeNames={
                "#status": "status"
            },
            ExpressionAttributeValues={
                ":status": TicketStatus.WAITING_FOR_INPUT.value,
                ":selected_workflow_id": selected_workflow_id,
                ":selected_workflow_type": selected_workflow_type,
                ":extracted_fields": extracted_fields,
                ":missing_fields": missing_fields,
                ":missing_field_questions": missing_field_questions,
                ":updated_at": updated_at,
            },
            ReturnValues="ALL_NEW",
        )

        return self._item_to_ticket(response["Attributes"])

    except ClientError as error:

```

```

        if error.response["Error"]["Code"] ==
"ConditionalCheckFailedException":
            return None
        raise

def _ticket_to_item(self, ticket: TicketResponse) -> dict:
    return {
        "ticket_id": ticket.ticket_id,
        "user_id": ticket.user_id,
        "message": ticket.message,
        "source": ticket.source,
        "status": ticket.status.value,
        "created_at": ticket.created_at,
        "updated_at": ticket.updated_at,
        "selected_workflow_id": ticket.selected_workflow_id,
        "selected_workflow_type": ticket.selected_workflow_type,
        "workflow_run_id": ticket.workflow_run_id,
        "extracted_fields": ticket.extracted_fields,
        "missing_fields": ticket.missing_fields,
        "missing_field_questions": ticket.missing_field_questions,
    }

def _item_to_ticket(self, item: dict) -> TicketResponse:
    return TicketResponse(
        ticket_id=item["ticket_id"],
        user_id=item["user_id"],
        message=item["message"],
        source=item.get("source", "unknown"),
        status=TicketStatus(item["status"]),
        created_at=item["created_at"],
        updated_at=item["updated_at"],
        selected_workflow_id=item.get("selected_workflow_id"),
        selected_workflow_type=item.get("selected_workflow_type"),
        workflow_run_id=item.get("workflow_run_id"),
        extracted_fields=item.get("extracted_fields", {}),
        missing_fields=item.get("missing_fields", []),
        missing_field_questions=item.get("missing_field_questions", []),
    )

```

Step 9: Create Workflow Run Repository

Create:

```
app/repositories/workflow_run_repository.py
```

Add:

```
from typing import Optional

import boto3
from botocore.exceptions import ClientError

from app.config import AWS_REGION, WORKFLOW_RUNS_TABLE_NAME
from app.models.workflow_run_models import (
    WorkflowRunResponse,
    WorkflowRunStatus,
)

class WorkflowRunRepository:
    def __init__(self):
        dynamodb = boto3.resource("dynamodb", region_name=AWS_REGION)
        self.table = dynamodb.Table(WORKFLOW_RUNS_TABLE_NAME)

    def save(self, workflow_run: WorkflowRunResponse) -> WorkflowRunResponse:
        self.table.put_item(Item=self._workflow_run_to_item(workflow_run))
        return workflow_run

    def get_by_id(self, workflow_run_id: str) -> Optional[WorkflowRunResponse]:
        response = self.table.get_item(
            Key={
                "workflow_run_id": workflow_run_id
            }
        )

        item = response.get("Item")

        if item is None:
            return None

        return self._item_to_workflow_run(item)

    def update_status(
        self,
        workflow_run_id: str,
        status: WorkflowRunStatus,
        updated_at: str,
        error_message: Optional[str] = None,
    ) -> Optional[WorkflowRunResponse]:
```

```

update_expression = "SET #status = :status, updated_at = :updated_at"
expression_attribute_names = {
    "#status": "status"
}
expression_attribute_values = {
    ":status": status.value,
    ":updated_at": updated_at,
}

if error_message is not None:
    update_expression += ", error_message = :error_message"
    expression_attribute_values[":error_message"] = error_message

try:
    response = self.table.update_item(
        Key={
            "workflow_run_id": workflow_run_id
        },
        UpdateExpression=update_expression,
        ConditionExpression="attribute_exists(workflow_run_id)",
        ExpressionAttributeNames=expression_attribute_names,
        ExpressionAttributeValues=expression_attribute_values,
        ReturnValues="ALL_NEW",
    )

    return self._item_to_workflow_run(response["Attributes"])

except ClientError as error:
    if error.response["Error"]["Code"] ==
"ConditionalCheckFailedException":
        return None
    raise

def _workflow_run_to_item(self, workflow_run: WorkflowRunResponse) -> dict:
    item = {
        "workflow_run_id": workflow_run.workflow_run_id,
        "ticket_id": workflow_run.ticket_id,
        "workflow_id": workflow_run.workflow_id,
        "workflow_type": workflow_run.workflow_type,
        "status": workflow_run.status.value,
        "workflow_input": workflow_run.workflow_input,
        "created_at": workflow_run.created_at,
        "updated_at": workflow_run.updated_at,
    }

    if workflow_run.error_message is not None:
        item["error_message"] = workflow_run.error_message

```

```

        return item

def _item_to_workflow_run(self, item: dict) -> WorkflowRunResponse:
    return WorkflowRunResponse(
        workflow_run_id=item["workflow_run_id"],
        ticket_id=item["ticket_id"],
        workflow_id=item["workflow_id"],
        workflow_type=item["workflow_type"],
        status=WorkflowRunStatus(item["status"]),
        workflow_input=item.get("workflow_input", {}),
        created_at=item["created_at"],
        updated_at=item["updated_at"],
        error_message=item.get("error_message"),
    )

```

Step 10: Create Workflow Transaction Repository

Create:

```
app/repositories/workflow_transaction_repository.py
```

Add:

```

import boto3
from boto3.dynamodb.types import TypeSerializer

from app.config import (
    AWS_REGION,
    TICKETS_TABLE_NAME,
    WORKFLOW_RUNS_TABLE_NAME,
)
from app.models.ticket_models import TicketStatus
from app.models.workflow_run_models import WorkflowRunResponse,
WorkflowRunStatus

class WorkflowTransactionRepository:
    def __init__(self):
        self.client = boto3.client("dynamodb", region_name=AWS_REGION)
        self.serializer = TypeSerializer()

    def create_workflow_run_and_attach_ticket(
        self,

```

```

        workflow_run: WorkflowRunResponse,
        extracted_fields: dict,
        updated_at: str,
    ) -> None:
        workflow_run_item = {
            "workflow_run_id": workflow_run.workflow_run_id,
            "ticket_id": workflow_run.ticket_id,
            "workflow_id": workflow_run.workflow_id,
            "workflow_type": workflow_run.workflow_type,
            "status": workflow_run.status.value,
            "workflow_input": workflow_run.workflow_input,
            "created_at": workflow_run.created_at,
            "updated_at": workflow_run.updated_at,
        }

        self.client.transact_write_items(
            TransactItems=[
                {
                    "Put": {
                        "TableName": WORKFLOW_RUNS_TABLE_NAME,
                        "Item": self._serialize_item(workflow_run_item),
                        "ConditionExpression": (
                            "attribute_not_exists(workflow_run_id)"
                        ),
                    },
                },
                {
                    "Update": {
                        "TableName": TICKETS_TABLE_NAME,
                        "Key": self._serialize_item(
                            {
                                "ticket_id": workflow_run.ticket_id
                            }
                        ),
                        "UpdateExpression": (
                            "SET #status = :status, "
                            "selected_workflow_id = :workflow_id, "
                            "selected_workflow_type = :workflow_type, "
                            "workflow_run_id = :workflow_run_id, "
                            "extracted_fields = :extracted_fields, "
                            "missing_fields = :missing_fields, "
                        ),
                        "missing_field_questions = :missing_field_questions, "
                        "updated_at = :updated_at"
                    ),
                    "ConditionExpression": "attribute_exists(ticket_id)",
                    "ExpressionAttributeNames": {
                        "#status": "status"
                    }
                }
            ]
        )

```

```

        },
        "ExpressionAttributeValues": self._serialize_item(
            {
                ":status": TicketStatus.WORKFLOW_CREATED.value,
                ":workflow_id": workflow_run.workflow_id,
                ":workflow_type": workflow_run.workflow_type,
                ":workflow_run_id":
workflow_run.workflow_run_id,
                ":extracted_fields": extracted_fields,
                ":missing_fields": [],
                ":missing_field_questions": [],
                ":updated_at": updated_at,
            }
        ),
    },
]
)

def mark_workflow_queued(
    self,
    ticket_id: str,
    workflow_run_id: str,
    updated_at: str,
) -> None:
    self.client.transact_write_items(
        TransactItems=[
            {
                "Update": {
                    "TableName": WORKFLOW_RUNS_TABLE_NAME,
                    "Key": self._serialize_item(
                        {
                            "workflow_run_id": workflow_run_id
                        }
                    ),
                    "UpdateExpression": (
                        "SET #status = :status, updated_at = :updated_at"
                    ),
                    "ConditionExpression":
"attribute_exists(workflow_run_id)",
                    "ExpressionAttributeNames": {
                        "#status": "status"
                    },
                    "ExpressionAttributeValues": self._serialize_item(
                        {
                            ":status": WorkflowRunStatus.QUEUED.value,
                            ":updated_at": updated_at,
                        }
                    )
                }
            ]
        )

```

```

        ),
    },
    {
        "Update": {
            "TableName": TICKETS_TABLE_NAME,
            "Key": self._serialize_item(
                {
                    "ticket_id": ticket_id
                }
            ),
            "UpdateExpression": (
                "SET #status = :status, updated_at = :updated_at"
            ),
            "ConditionExpression": "attribute_exists(ticket_id)",
            "ExpressionAttributeNames": {
                "#status": "status"
            },
            "ExpressionAttributeValues": self._serialize_item(
                {
                    ":status": TicketStatus.WORKFLOW_QUEUED.value,
                    ":updated_at": updated_at,
                }
            ),
        },
    },
]

)

def mark_workflow_failed(
    self,
    ticket_id: str,
    workflow_run_id: str,
    error_message: str,
    updated_at: str,
) -> None:
    self.client.transact_write_items(
        TransactItems=[
            {
                "Update": {
                    "TableName": WORKFLOW_RUNS_TABLE_NAME,
                    "Key": self._serialize_item(
                        {
                            "workflow_run_id": workflow_run_id
                        }
                    ),
                    "UpdateExpression": (
                        "SET #status = :status, "

```

```

        "error_message = :error_message, "
        "updated_at = :updated_at"
    ),
    "ConditionExpression":
"attribute_exists(workflow_run_id)",
    "ExpressionAttributeNames": {
        "#status": "status"
    },
    "ExpressionAttributeValues": self._serialize_item(
        {
            ":status": WorkflowRunStatus.FAILED.value,
            ":error_message": error_message,
            ":updated_at": updated_at,
        }
    ),
    },
    {
        "Update": {
            "TableName": TICKETS_TABLE_NAME,
            "Key": self._serialize_item(
                {
                    "ticket_id": ticket_id
                }
            ),
            "UpdateExpression": (
                "SET #status = :status, updated_at = :updated_at"
            ),
            "ConditionExpression": "attribute_exists(ticket_id)",
            "ExpressionAttributeNames": {
                "#status": "status"
            },
            "ExpressionAttributeValues": self._serialize_item(
                {
                    ":status": TicketStatus.FAILED.value,
                    ":updated_at": updated_at,
                }
            ),
        },
    },
    ],
)

def _serialize_item(self, item: dict) -> dict:
    return {
        key: self.serializer.serialize(value)
        for key, value in item.items()
    }

```

Step 11: Create Workflow Run Service

Create:

```
app/services/workflow_run_service.py
```

Add:

```
from datetime import datetime, timezone
from typing import Dict, Optional
from uuid import uuid4

from app.models.workflow_run_models import (
    WorkflowRunResponse,
    WorkflowRunStatus,
)
from app.repositories.workflow_run_repository import WorkflowRunRepository

class WorkflowRunService:
    def __init__(self):
        self.workflow_run_repository = WorkflowRunRepository()

    def build_workflow_run(
        self,
        ticket_id: str,
        workflow_id: str,
        workflow_type: str,
        workflow_input: Dict[str, str],
    ) -> WorkflowRunResponse:
        now = self._current_time_iso()

        return WorkflowRunResponse(
            workflow_run_id=f"run_{uuid4().hex[:8]}",
            ticket_id=ticket_id,
            workflow_id=workflow_id,
            workflow_type=workflow_type,
            status=WorkflowRunStatus.CREATED,
            workflow_input=workflow_input,
            created_at=now,
            updated_at=now,
        )
```

```
def get_workflow_run(
    self,
    workflow_run_id: str,
) -> Optional[WorkflowRunResponse]:
    return self.workflow_run_repository.get_by_id(workflow_run_id)

def _current_time_iso(self) -> str:
    return datetime.now(timezone.utc).isoformat()
```

Step 12: Create SQS Service

Create:

```
app/services/sqs_service.py
```

Add:

```
import json

import boto3

from app.config import AWS_REGION, WORKFLOW_QUEUE_URL

class SQSService:
    def __init__(self):
        self.sqs = boto3.client("sqs", region_name=AWS_REGION)
        self.queue_url = WORKFLOW_QUEUE_URL

    def send_workflow_run_message(self, workflow_run_id: str) -> None:
        if not self.queue_url:
            raise RuntimeError("WORKFLOW_QUEUE_URL is not set.")

        message_body = {
            "workflow_run_id": workflow_run_id
        }

        self.sqs.send_message(
            QueueUrl=self.queue_url,
            MessageBody=json.dumps(message_body),
        )
```

Step 13: Update Ticket Service

Open:

```
app/services/ticket_service.py
```

Replace it with:

```
from datetime import datetime, timezone
from typing import List, Optional
from uuid import uuid4

from app.models.ticket_models import (
    CreateTicketRequest,
    TicketResponse,
    TicketStatus,
)
from app.repositories.ticket_repository import TicketRepository
from app.repositories.workflow_transaction_repository import (
    WorkflowTransactionRepository,
)
from app.services.llm_workflow_selector_service import (
    llm_workflow_selector_service,
)
from app.services.sqs_service import SQSService
from app.services.workflow_run_service import WorkflowRunService
from app.models.workflow_selector_models import WorkflowSelectionRequest

class TicketService:
    def __init__(self):
        self.ticket_repository = TicketRepository()
        self.workflow_run_service = WorkflowRunService()
        self.workflow_transaction_repository = WorkflowTransactionRepository()
        self.sqs_service = SQSService()

    def create_ticket(self, request: CreateTicketRequest) -> TicketResponse:
        now = self._current_time_iso()

        ticket = TicketResponse(
            ticket_id=f"ticket_{uuid4().hex[:8]}",
            user_id=request.user_id,
            message=request.message,
            source=request.source or "unknown",
            status=TicketStatus.CREATED,
```

```

        created_at=now,
        updated_at=now,
    )

    saved_ticket = self.ticket_repository.save(ticket)

    selection = llm_workflow_selector_service.select_workflow(
        WorkflowSelectionRequest(
            message=saved_ticket.message,
            user_id=saved_ticket.user_id,
            source=saved_ticket.source,
        )
    )

    if not selection.is_ready_for_workflow:
        waiting_ticket = self.ticket_repository.mark_waiting_for_input(
            ticket_id=saved_ticket.ticket_id,
            selected_workflow_id=selection.selected_workflow_id,
            selected_workflow_type=selection.selected_workflow_type,
            extracted_fields=selection.extracted_fields,
            missing_fields=selection.missing_fields,
            missing_field_questions=selection.missing_field_questions,
            updated_at=self._current_time_iso(),
        )

        if waiting_ticket is None:
            raise RuntimeError("Failed to update ticket to
WAITING_FOR_INPUT.")

        return waiting_ticket

    workflow_run = self.workflow_run_service.build_workflow_run(
        ticket_id=saved_ticket.ticket_id,
        workflow_id=selection.selected_workflow_id,
        workflow_type=selection.selected_workflow_type,
        workflow_input=selection.extracted_fields,
    )

    self.workflow_transaction_repository.create_workflow_run_and_attach_ticket(
        workflow_run=workflow_run,
        extracted_fields=selection.extracted_fields,
        updated_at=self._current_time_iso(),
    )

    try:
        self.sqs_service.send_workflow_run_message(
            workflow_run_id=workflow_run.workflow_run_id

```

```

    )

    self.workflow_transaction_repository.mark_workflow_queued(
        ticket_id=saved_ticket.ticket_id,
        workflow_run_id=workflow_run.workflow_run_id,
        updated_at=self._current_time_iso(),
    )

except Exception as error:
    self.workflow_transaction_repository.mark_workflow_failed(
        ticket_id=saved_ticket.ticket_id,
        workflow_run_id=workflow_run.workflow_run_id,
        error_message=str(error),
        updated_at=self._current_time_iso(),
    )

    failed_ticket =
self.ticket_repository.get_by_id(saved_ticket.ticket_id)

    if failed_ticket is None:
        raise

    return failed_ticket

queued_ticket = self.ticket_repository.get_by_id(saved_ticket.ticket_id)

if queued_ticket is None:
    raise RuntimeError("Ticket not found after workflow queueing.")

return queued_ticket

def get_ticket(self, ticket_id: str) -> Optional[TicketResponse]:
    return self.ticket_repository.get_by_id(ticket_id)

def list_tickets(self) -> List[TicketResponse]:
    return self.ticket_repository.list_all()

def update_ticket_status(
    self,
    ticket_id: str,
    status: TicketStatus,
) -> Optional[TicketResponse]:
    return self.ticket_repository.update_status(
        ticket_id=ticket_id,
        status=status,
        updated_at=self._current_time_iso(),
    )

```

```
def _current_time_iso(self) -> str:
    return datetime.now(timezone.utc).isoformat()

ticket_service = TicketService()
```

Step 14: Create Workflow Run Router

Create:

```
app/routers/workflow_run_router.py
```

Add:

```
from fastapi import APIRouter, HTTPException, status

from app.models.workflow_run_models import WorkflowRunResponse
from app.services.workflow_run_service import WorkflowRunService

router = APIRouter(
    prefix="/workflow-runs",
    tags=["Workflow Runs"],
)

workflow_run_service = WorkflowRunService()

@router.get(
   ("/{workflow_run_id}",
    response_model=WorkflowRunResponse,
)
def get_workflow_run(workflow_run_id: str):
    workflow_run = workflow_run_service.get_workflow_run(workflow_run_id)

    if workflow_run is None:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Workflow run not found",
        )
```

```
return workflow_run
```

Step 15: Update app/main.py

```
from fastapi import FastAPI

from app.routers.ticket_router import router as ticket_router
from app.routers.workflow_definition_router import (
    router as workflow_definition_router,
)
from app.routers.workflow_selector_router import (
    router as workflow_selector_router,
)
from app.routers.workflow_run_router import router as workflow_run_router

app = FastAPI(
    title="AI Ticket Workflow Platform",
    description="A backend platform for client issue triage and workflow automation.",
    version="0.4.0",
)

app.include_router(ticket_router)
app.include_router(workflow_definition_router)
app.include_router(workflow_selector_router)
app.include_router(workflow_run_router)

@app.get("/health")
def health_check():
    return {
        "status": "ok"
    }
```

Step 16: Run Server

```
uvicorn app.main:app --reload
```

Open:

```
http://127.0.0.1:8000/docs
```

Step 17: Test Successful Workflow Selection

Call:

```
POST /tickets
```

Request body:

```
{
  "user_id": "user_123",
  "message": "I did not get an alert when TSLA dropped below 200.",
  "source": "mobile_app"
}
```

Expected response:

```
{
  "ticket_id": "ticket_xxxxxxx",
  "user_id": "user_123",
  "message": "I did not get an alert when TSLA dropped below 200.",
  "source": "mobile_app",
  "status": "WORKFLOW_QUEUED",
  "created_at": "...",
  "updated_at": "...",
  "selected_workflow_id": "wf_price_alert_bug_v1",
  "selected_workflow_type": "PRICE_ALERT_BUG",
  "workflow_run_id": "run_xxxxxxx",
  "extracted_fields": {
    "alert_issue": "...",
    "symbol": "TSLA",
  }
}
```

```
    "target_price": "200"
  },
  "missing_fields": [],
  "missing_field_questions": []
}
```

Step 18: Verify Workflow Run

Use the returned `workflow_run_id`.

Call:

```
GET /workflow-runs/{workflow_run_id}
```

Expected response:

```
{
  "workflow_run_id": "run_xxxxxxx",
  "ticket_id": "ticket_xxxxxxx",
  "workflow_id": "wf_price_alert_bug_v1",
  "workflow_type": "PRICE_ALERT_BUG",
  "status": "QUEUED",
  "workflow_input": {
    "alert_issue": "...",
    "symbol": "TSLA",
    "target_price": "200"
  },
  "created_at": "...",
  "updated_at": "...",
  "error_message": null
}
```

Step 19: Verify SQS Message

Open AWS Console.

Go to:

SQS

- ticket-workflow-queue
- Send and receive messages
- Poll for messages

Expected message body:

```
{
  "workflow_run_id": "run_xxxxxxx"
}
```

Step 20: Test Missing Required Field

Call:

```
POST /tickets
```

Request body:

```
{
  "user_id": "user_123",
  "message": "The stock price is wrong.",
  "source": "mobile_app"
}
```

Expected response:

```
{
  "ticket_id": "ticket_xxxxxxx",
  "user_id": "user_123",
  "message": "The stock price is wrong.",
  "source": "mobile_app",
  "status": "WAITING_FOR_INPUT",
  "selected_workflow_id": "wf_stock_data_issue_v1",
  "selected_workflow_type": "STOCK_DATA_ISSUE",
  "workflow_run_id": null,
  "extracted_fields": {
    "issue_description": "stock price is wrong"
  },
}
```

```
"missing_fields": [  
  "symbol"  
],  
"missing_field_questions": [  
  "Which stock symbol has the issue?"  
]  
}
```

Step 21: Commit Code

```
git status  
git add .  
git commit -m "Integrate ticket service with AI selector and SQS producer"  
git push origin main
```

Acceptance Criteria

The assignment is complete when:

1. `WorkflowRuns` DynamoDB table exists.
2. `ticket-workflow-queue` SQS queue exists.
3. `POST /tickets` creates a ticket.
4. `POST /tickets` calls the LLM workflow selector.
5. If the selector is ready, a workflow run is created.
6. The ticket stores `selected_workflow_id`.
7. The ticket stores `selected_workflow_type`.
8. The ticket stores `workflow_run_id`.
9. The workflow run stores `workflow_input`.
10. The SQS message contains only `workflow_run_id`.
11. A successful ticket ends with status `WORKFLOW_QUEUED`.
12. A successful workflow run ends with status `QUEUED`.
13. A ticket with missing required fields ends with status `WAITING_FOR_INPUT`.
14. Missing fields and missing field questions are returned in the ticket response.

Questions

1. Why does the SQS message only contain `workflow_run_id`?
2. Why do we create a `WorkflowRun` instead of only using `Ticket`?

3. What is the difference between `Ticket.status` and `WorkflowRun.status`?
4. Why do we use a DynamoDB transaction when creating a workflow run and updating the ticket?
5. Why can SQS send not be part of the DynamoDB transaction?
6. What status should the ticket have after the workflow run is queued?
7. What status should the workflow run have after the SQS message is sent?
8. What should happen when the AI selector is missing required fields?